

Script-Controlled ACL – Restrict Record Access Based on Field Value

Project Description

The **Script-Controlled ACL – Restrict Record Access Based on Field Value** project is a ServiceNow-based access-control solution designed to provide controlled and secure access to records based on user roles and field values.

The project implements Access Control Lists (ACLs) for **READ, CREATE, WRITE, and DELETE** operations. The **READ** access is controlled using the **Branch** field, while different roles are used to control the various record operations.

The solution uses a custom **Institution Details** table and verifies access through user impersonation. Administrators are provided with full access, while other users receive permissions according to their assigned roles and the configured ACL conditions.

Project Information

Team ID: SWTID-2026-5919

Team Size: 5

Team Leader: V. Abitha

Team Members

- V. Abitha
- T. Archanadevi
- T. Akalya
- **M.Kishor Banu**
- T. Atchaya

EDITED: Team member name changed from ~~M. Kishor Banu~~ to **M.Kishor Banu**.

Project Workflow

Milestone 1: User and Role Configuration

The first milestone establishes the users and roles required for ACL testing.

Activity 1.1: Create Test User

Create a test user named **EEE User** for verifying the configured access-control rules.

Activity 1.2: Create Roles

Create the following custom roles:

- bb1
- bb2
- bb3
- bb4

Assign these roles to the EEE User for testing the different ACL operations.

Milestone 2: Custom Table Creation

The second milestone creates the custom table used by the ACL implementation.

Activity 2.1: Create Institution Details Table

Label: Institution Details

Name: u_institution_details

Extends: False

Activity 2.2: Configure Table Fields

Field	Type
-------	------

Student Roll Number	Auto Number
Student Name	Reference – User
Faculty Name	Reference – User
Branch	Choice
Email	String
Phone Number	String
Description	Multi String

The **Branch** field contains the following choices:

- ECE
- EEE
- CSE

The Branch field is used as part of the access-control requirement.

Milestone 3: READ ACL Configuration

The third milestone implements the READ ACL for the Institution Details table.

Activity 3.1: Open Access Control

Navigate to: **System Security → Access Control (ACL)**
Elevate the user to the **security_admin** role when required.

Activity 3.2: Configure READ ACL

- **Type:** Record
- **Operation:** Read
- **Name:** u_institution_details
- **Active:** True
- **Advanced:** True
- **Required Role:** bb1
- **Data Condition:** Branch is EEE

Activity 3.3: Configure ACL Script

```
(function () {

    // Allow admin users full access
    if (gs.hasRole('admin')) {
        return true;
    }

    // Allow only EEE branch users to see EEE records
    if (gs.hasRole('bb1'))
    {
        return true;
    }

    // Deny access for all others
    return false;

})();
```

Activity 3.4: READ ACL Verification

The READ ACL is verified using user impersonation.

- EEE user with bb1 role.
- Users without the required role.
- Administrator access.

The expected behavior is that administrators can access records, while access for other users is controlled by the configured ACL conditions.

Milestone 4: CREATE ACL Configuration

The fourth milestone controls the creation of Institution Details records.

Activity 4.1: Configure CREATE ACL

Create a record-level CREATE ACL for u_institution_details.

- **Operation:** Create
- **Active:** True
- **Required Role:** bb2

Activity 4.2: CREATE ACL Verification

Verify that users with the required role can access the New functionality and create records according to the configured ACL. The CREATE permission is tested together with the existing READ access.

Milestone 5: WRITE ACL Configuration

The fifth milestone controls modification of Institution Details records.

Activity 5.1: Configure WRITE ACL

Create a record-level WRITE ACL for u_institution_details.

- **Operation:** Write
- **Active:** True
- **Required Role:** bb3

Activity 5.2: WRITE ACL Verification

Verify that users with the required WRITE permission can edit the appropriate Institution Details records. The testing confirms that the WRITE ACL works together with the existing READ and CREATE permissions.

Milestone 6: DELETE ACL Configuration

The sixth milestone controls deletion of Institution Details records.

Activity 6.1: Configure DELETE ACL

Create a record-level DELETE ACL for u_institution_details.

- **Operation:** Delete
- **Active:** True
- **Required Role:** bb4

Activity 6.2: DELETE ACL Verification

Verify that users with the required DELETE permission can delete the appropriate Institution Details records. The complete workflow is tested using the configured roles.

ACL Permission Workflow

ACL Operation	Role
READ	bb1
CREATE	bb2
WRITE	bb3
DELETE	bb4

This structure provides separate control over the major record operations.

Testing and Verification

The project is tested using ServiceNow user impersonation.

Test 1: READ Access

- The required user role is present.
- EEE branch records can be accessed according to the configured condition.
- Users without the required access are restricted.
- Administrators have full access.

Test 2: CREATE Access

Verify that users with the required CREATE permission can create Institution Details records.

Test 3: WRITE Access

Verify that users with the required WRITE permission can edit Institution Details records.

Test 4: DELETE Access

Verify that users with the required DELETE permission can delete Institution Details records.

Test 5: Complete ACL Workflow

Verify the combined behavior of:

- bb1
- bb2
- bb3
- bb4
- Administrator access

The final verification confirms the configured access-control behavior across the major record operations.

Demonstration

The final project demonstration covers:

- EEE User creation.
- Creation and assignment of the required roles.
- Institution Details custom table.
- Table fields and Branch choices.
- READ ACL configuration.
- CREATE ACL configuration.
- WRITE ACL configuration.
- DELETE ACL configuration.
- User impersonation.
- Verification of authorized and unauthorized access.
- Administrator access verification.

Conclusion

The **Script-Controlled ACL – Restrict Record Access Based on Field Value** project demonstrates how ServiceNow Access Control Lists can be used to secure records through role-based and field-based access control.

The project implements separate ACLs for **READ, CREATE, WRITE, and DELETE** operations. The custom Institution Details table provides the records on which the access rules are applied, while the Branch field and user roles are used to control access.

Through user impersonation and ACL verification, the project demonstrates controlled access to ServiceNow records and protection against unauthorized viewing, creation, modification, and deletion.

The project therefore provides practical understanding of ServiceNow ACL configuration, scripting, role-based permissions, field-based conditions, and access verification.